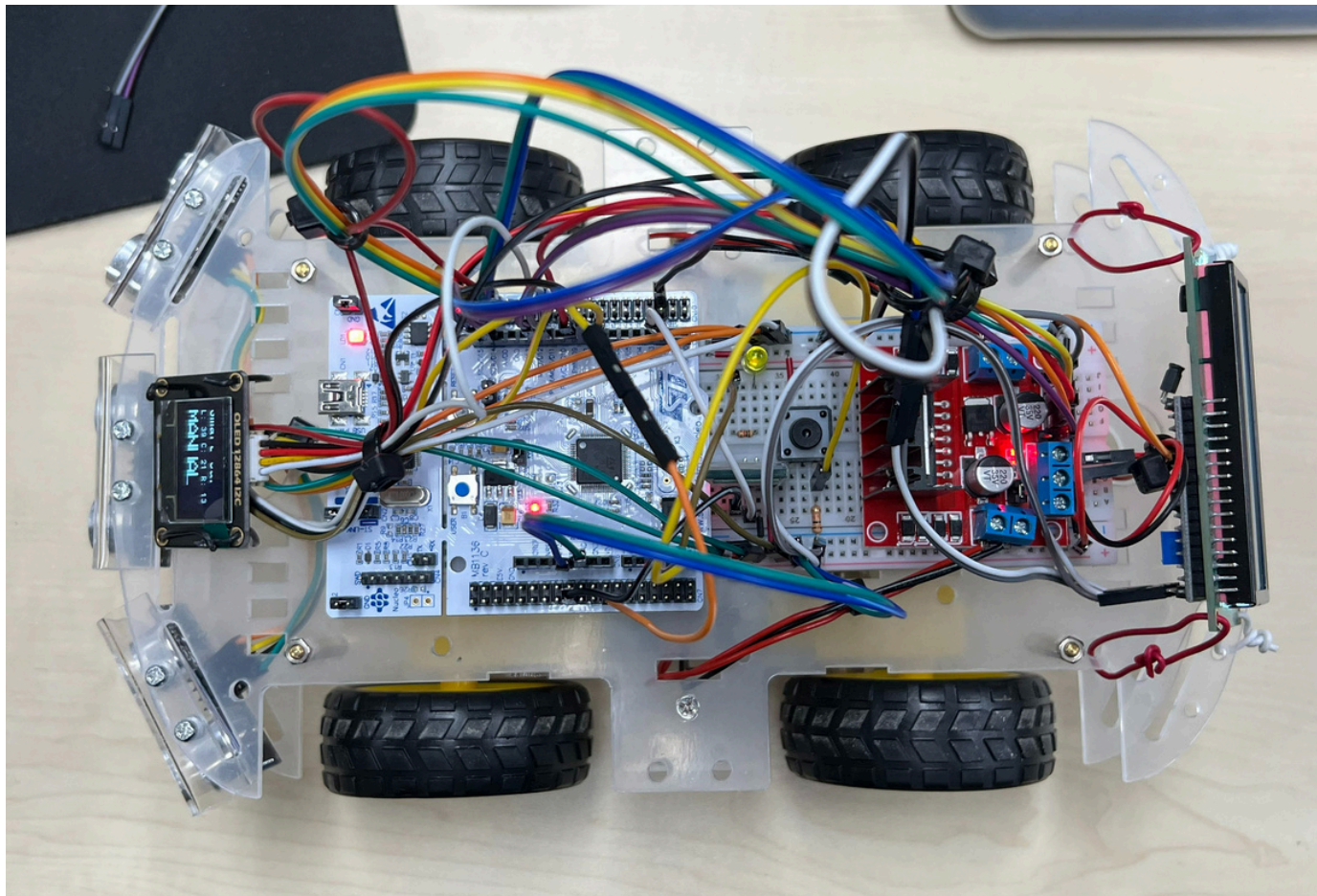


04. 자율주행 자동차 제어

프로젝트 개요



STM32 기반 자율주행 차량 시스템 설계 및 구현

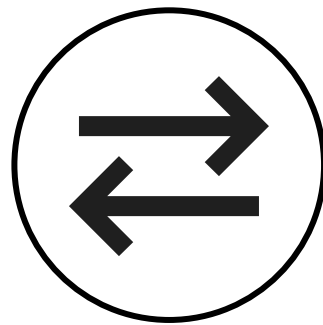
Button을 이용해 Manual mode와 Auto mode 변경

Bluetooth를 이용한 무선 조종

초음파 센서를 이용한 자율주행모드

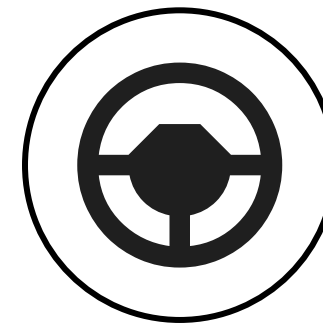
LCD/OLED를 통한 상태표시

프로젝트 목표



MODE 전환

수동 모드와 자동 모드 전환
수동 모드시 직접 조종 가능
자동 모드시 장애물을 인식해 회피 동작



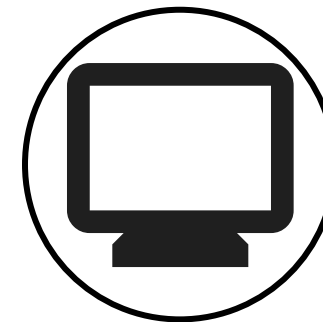
수동 제어

Bluetooth를 이용한 무선조종기능 탑재
Bluetooth module과 UART를 통해 STM32와 통신



센서 제어

초음파 센서를 이용한 거리 측정
장애물 탐지 및 회피 기동
PWM을 이용해 모터 제어



Display

LCD / OLED를 통한 상태 표시
현재 모드, 장애물까지와의 거리
현재 동작, 위험 표시 깜빡임

개발 환경

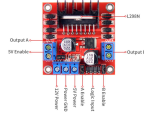
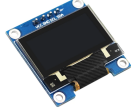


STM32F411re

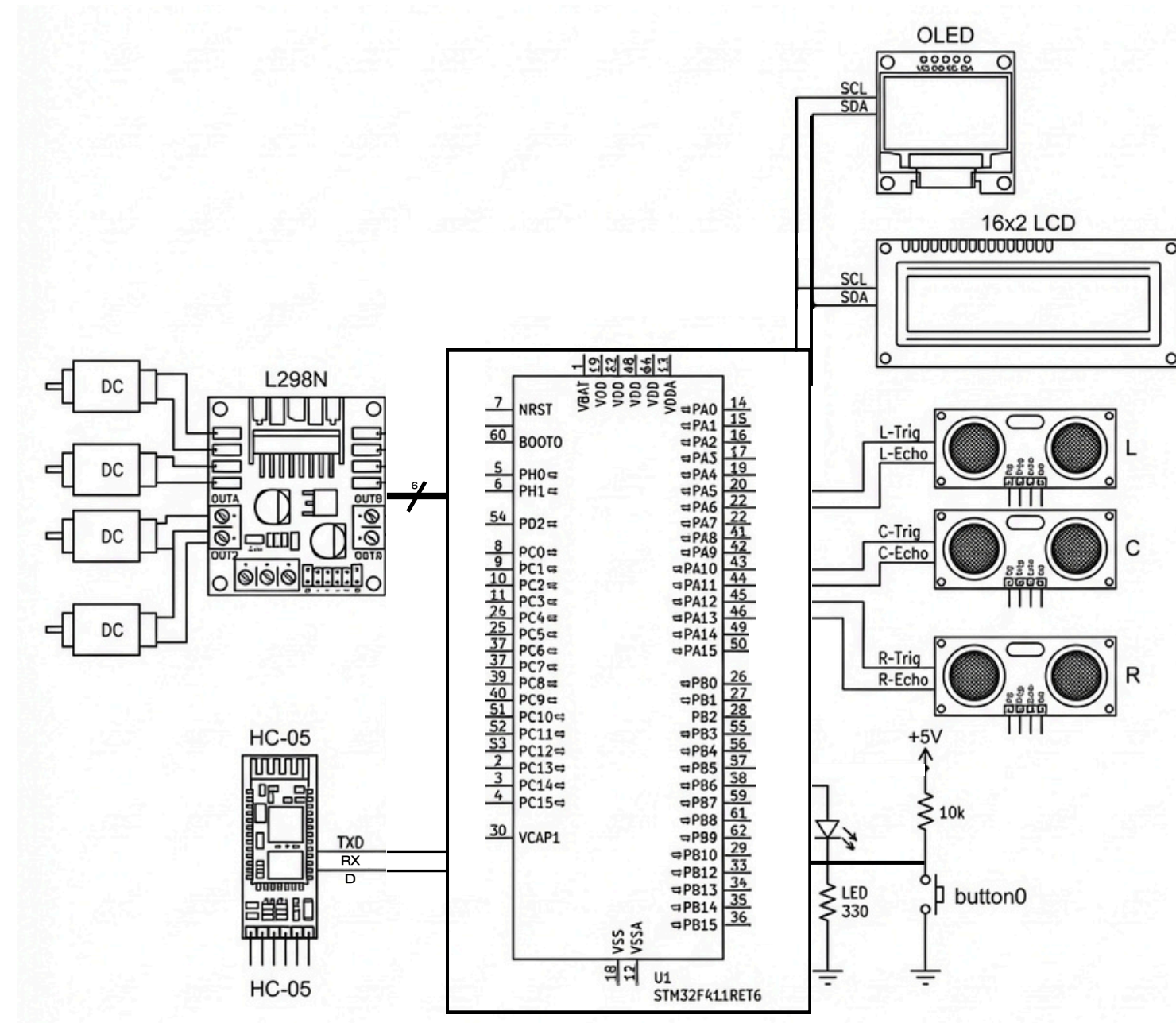
STM32
CubeIDE



구성 부품

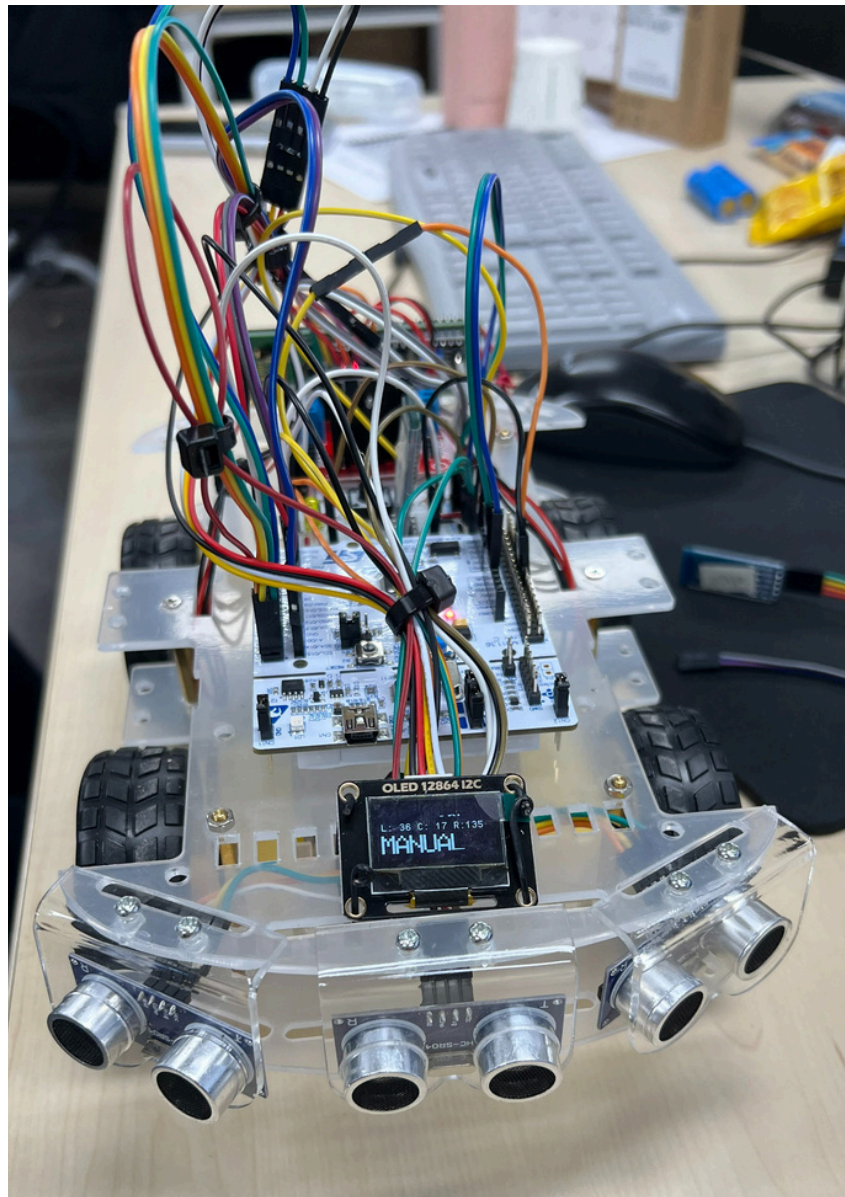
부품명	부품 사진	모델명 / 규격	입·출력 구분	수량	사용 목적
MCU 보드		STM32F411RE	제어부	1	전체 시스템 제어 및 연산
초음파 센서		HC-SR04	입력(Input)	3	전방 장애물 거리 측정
DC 모터		DC Motor	출력(Output)	4	차량 좌·우 구동
모터 드라이버		L298N	출력(Output)	1	모터 전류 증폭
OLED 디스플레이		SSD1306	출력(Output)	1	주행 상태 정보 표시
LCD 디스플레이		LCD1602	출력(Output)	1	모드 정보 표시
Bluetooth 모듈		HC-06	입·출력(Input/Output)	1	무선 제어 및 상태 전송

회로 결선도

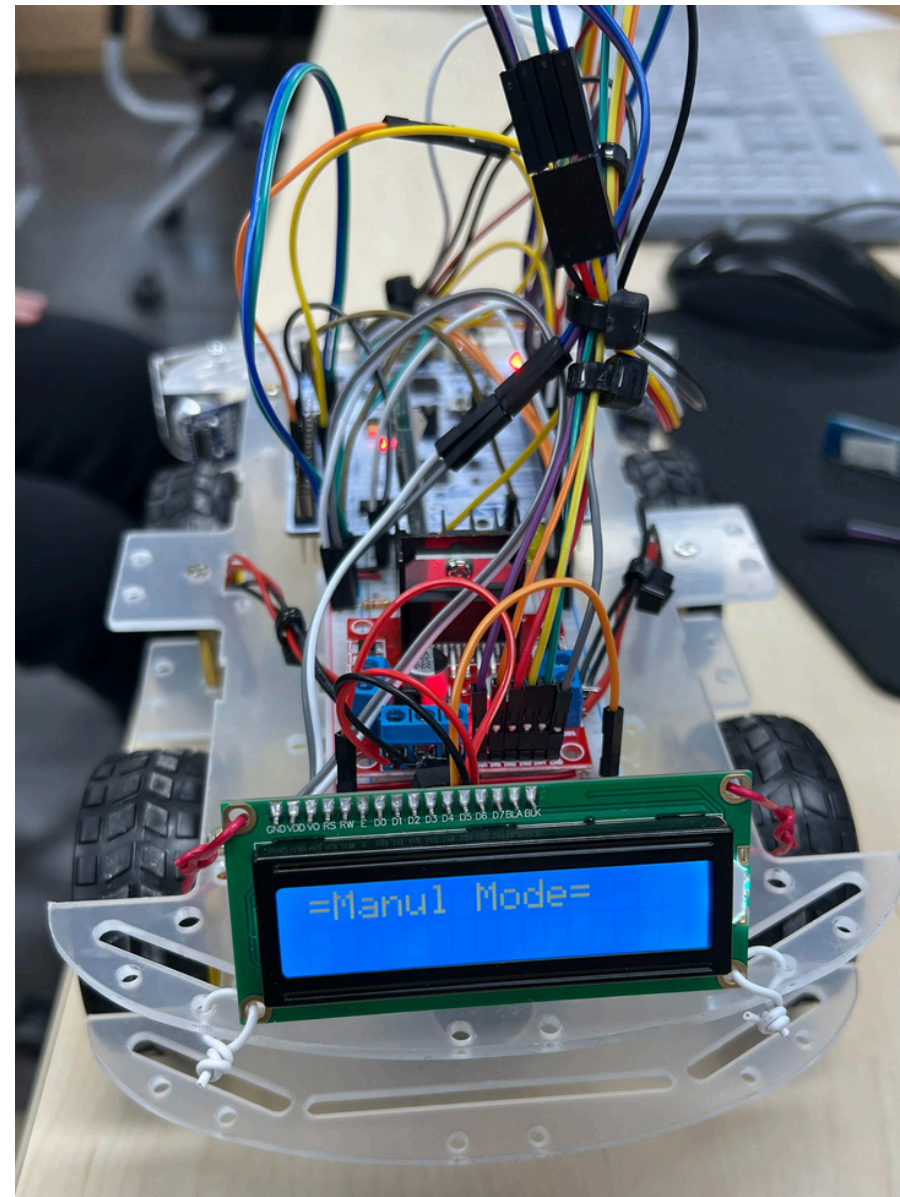


주변장치	연결 방식	역할
HC-SR04	GPIO	거리 측정
Button	GPIO	모드 전환
L298N	GPIO / PWM	모터 제어
HC-06	UART	무선 명령
LCD / OLED	I2C	상태 표시

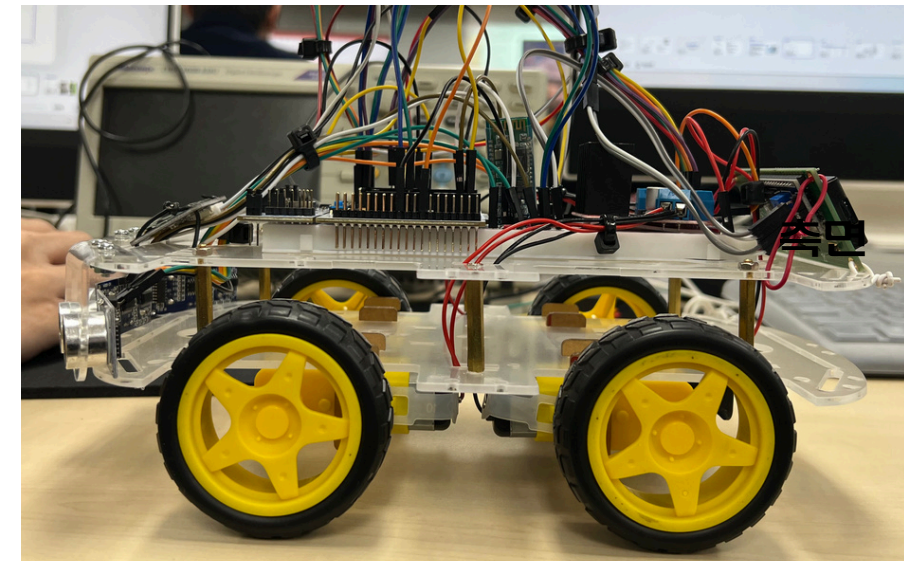
H/W 구성



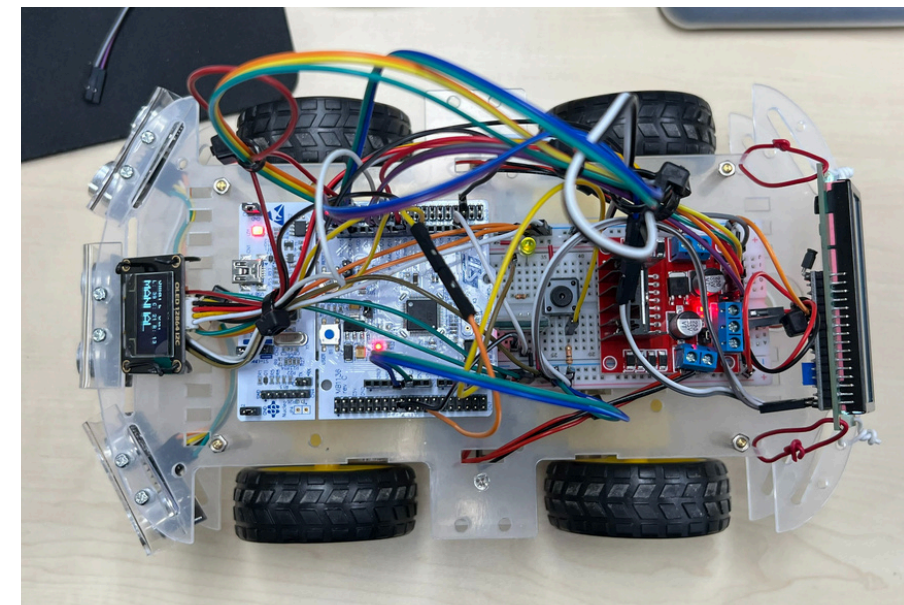
전
면



후
면



측면



상단

요소기술: PWM 기반 모터 제어

차량 동작	좌측 모터	우측 모터
전진	Forward	Forward
후진	Backward	Backward
좌회전	Slow	Forward
우회전	Forward	Slow
정지	Stop	Stop

요소기술: PWM 기반 모터 제어

(1) 개요

- PWM(Pulse Width Modulation)
: 디지털 신호의 듀티비(Duty Ratio)를 조절하여 아날로그 출력과 유사한 제어를 가능하게 하는 기술

(2) 용도

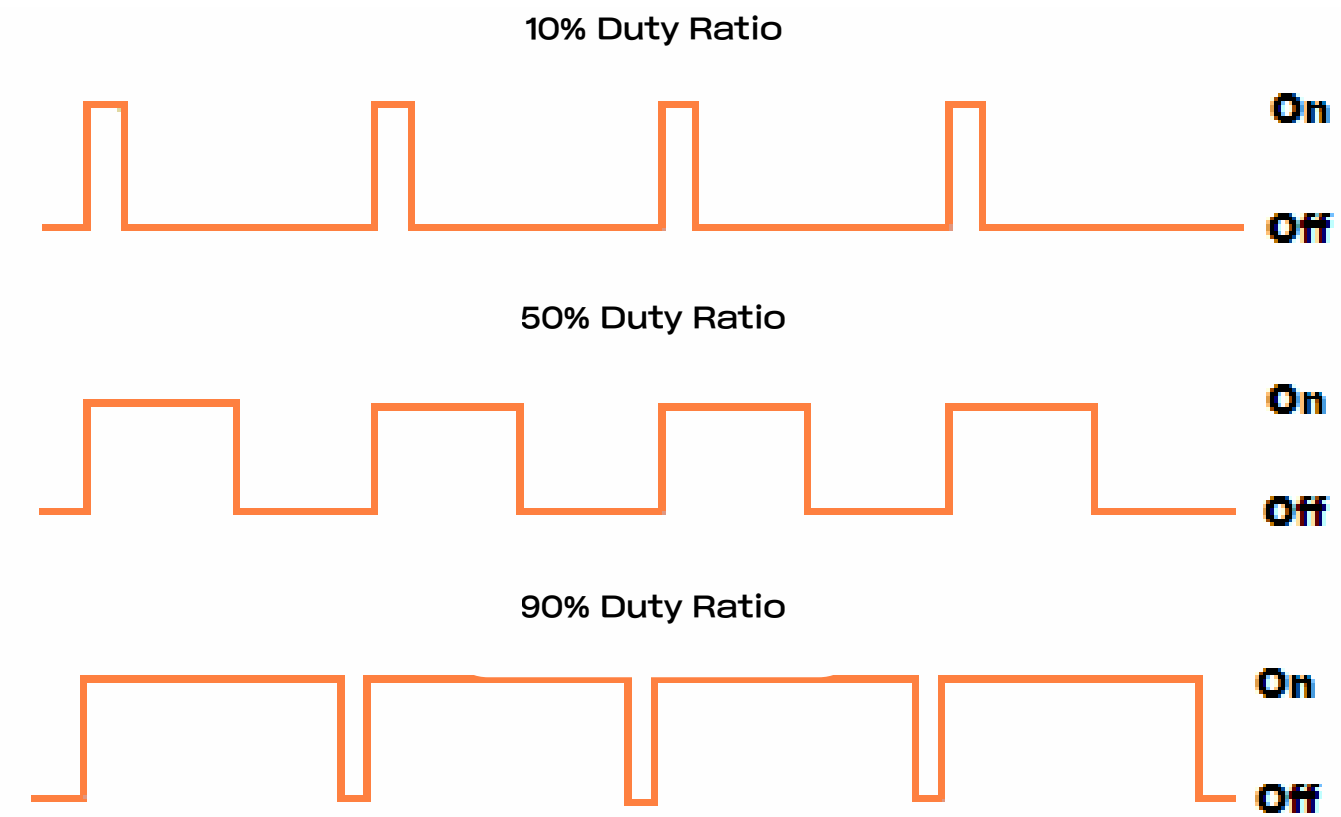
- DC 모터 속도 제어
- 듀티비 변화로 모터 인가 전압의 평균값을 조절 → 회전 속도 제어

(3) 동작 방식(도식 설명)

- 주기(Period) 내 HIGH 구간 비율(듀티비) 변화
- 듀티비 ↑ → 속도 증가 / 듀티비 ↓ → 속도 감소
- STM32 Timer + PWM Output 기능으로 신호 생성

(4) 특징 및 장단점

- 장점 : 디지털 핀만으로 정밀 제어 가능, 효율 높고 발열 적음
- 단점 : 듀티비 설정이 부적절하면 진동/소음 발생 가능



요소 기술: 초음파 센서 거리 측정

차량 동작	판단	동작
전방 거리 < Threshold	장애물 존재	정지/후진
좌측 거리 > 우측 거리	좌측 여유	좌회전
우측 거리 > 좌측 거리	우측 여유	우회전

요소 기술: 초음파 센서 거리 측정

(1) 개요

- 초음파 센서(Ultrasonic Sensor)
: 가청 주파수 이상의 초음파를 이용하여
물체까지의 거리를 비접촉 방식으로 측정하는 센서

(2) 용도

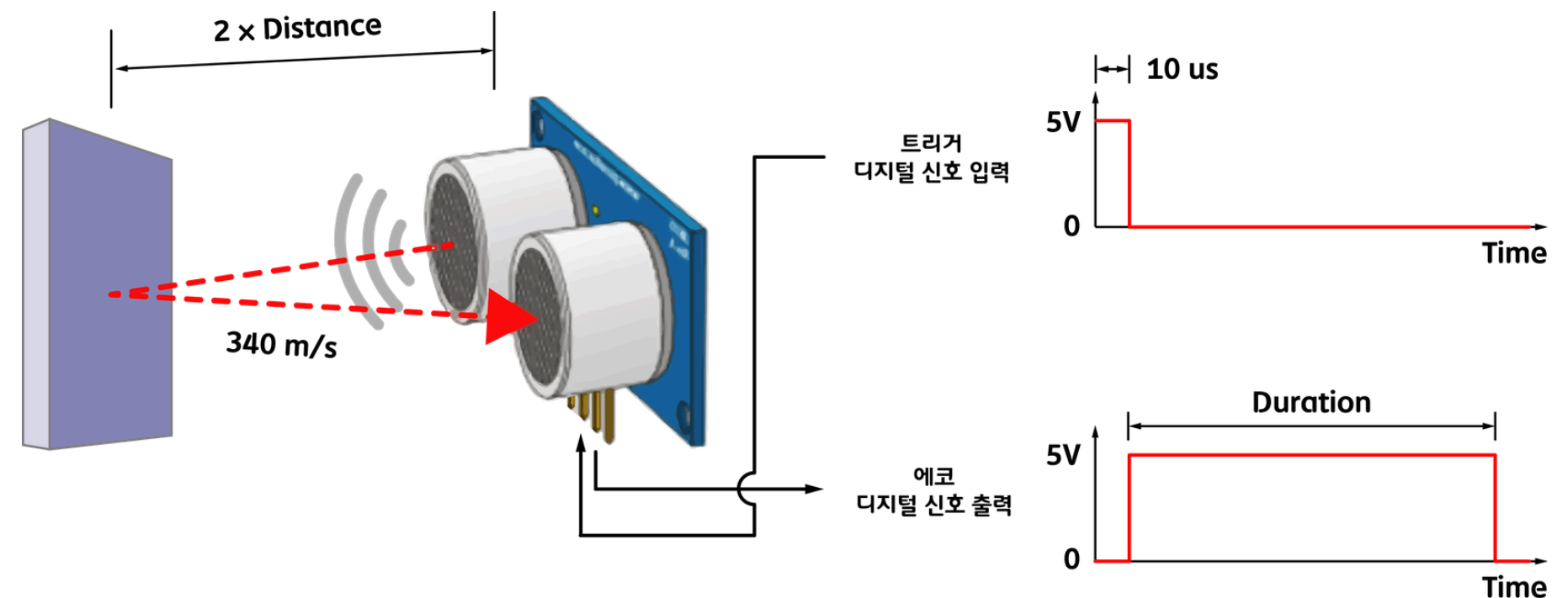
- 자율주행, 자동문, 로봇
- 거리 측정

(3) 동작 방식(도식 설명)

- TRIG 핀이 초음파를 송신
- ECHO 핀이 반사되어 돌아온 초음파를 수신
- ECHO 핀의 HIGH 유지 시간 = 초음파 왕복 시간

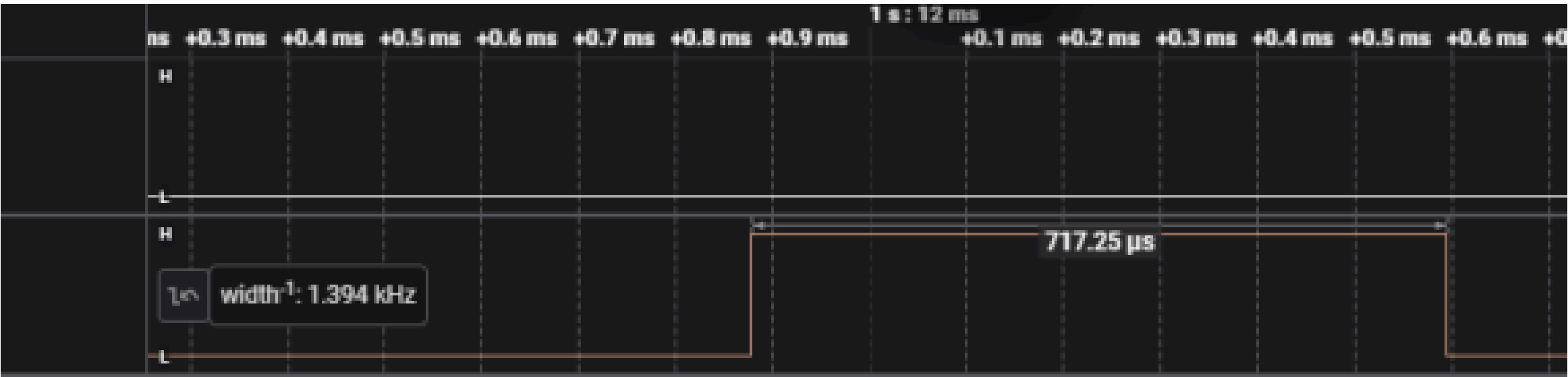
(4) 특징 및 장단점

- 장점 : 구조 단순/ 비접촉
- 단점 : 환경에 의한 음속 변화/ 물체의 특성에 의한 오차



$$\text{거리} = \frac{\text{초음파 왕복 시간} \times \text{음속}}{2}$$

요소 기술: 초음파 센서 거리 측정



요소 기술: Bluetooth

차량 동작	동작
F	전진
B	후진
L	좌회전
R	우회전
S	정지

요소 기술: Bluetooth

(1) 개요

- Bluetooth : 2.4GHz ISM 대역을 사용하는 근거리 무선 통신 기술
- 주파수 호핑(Frequency Hopping)으로 간섭 최소화

(2) 용도

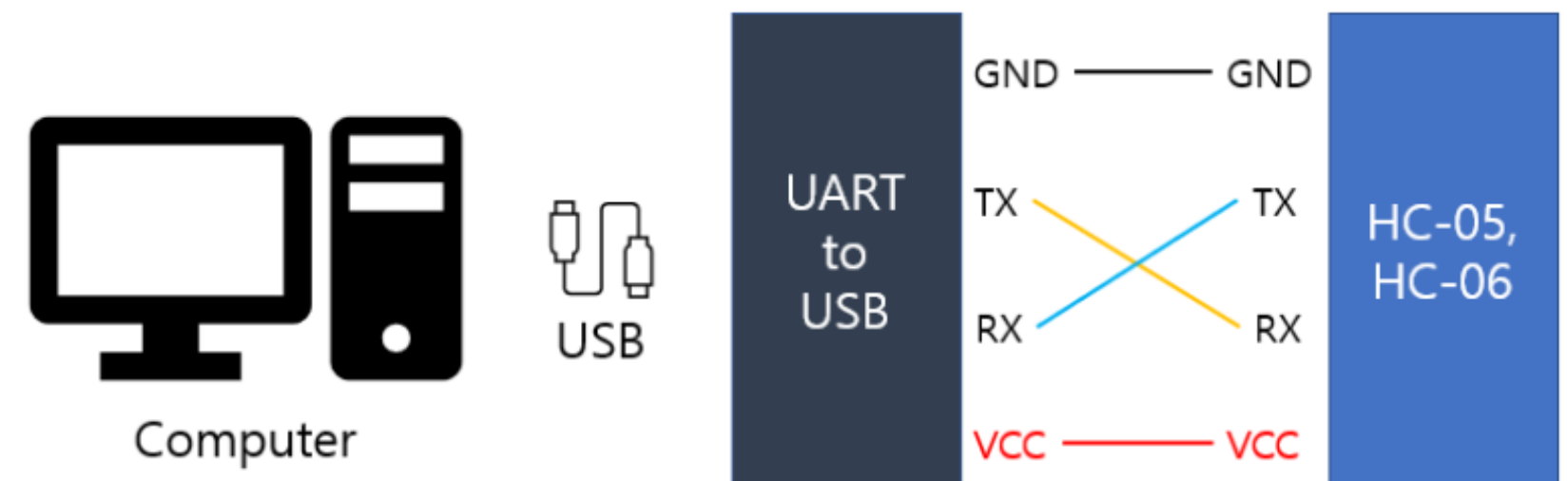
- 스마트폰 ↔ STM32 무선 명령 전달
- 수동 모드 제어 및 상태 정보 전송

(3) 동작 방식(도식 설명)

- Bluetooth 모듈(HC-06)이 UART로 MCU와 연결
- 주파수를 빠르게 변경하며 통신 → 안정성 확보
- 수신 명령을 MCU가 해석 후 제어 로직 수행

(4) 특징 및 장단점

- 장점 : 무선 제어로 편의성 향상, 구현 간단/안정적
- 단점 : 통신 거리 제한, 전파 간섭 환경 영향



요소 기술: UART

(1) 개요

- UART(Universal Asynchronous Receiver/Transmitter) : 비동기 직렬 통신 방식

(2) 용도

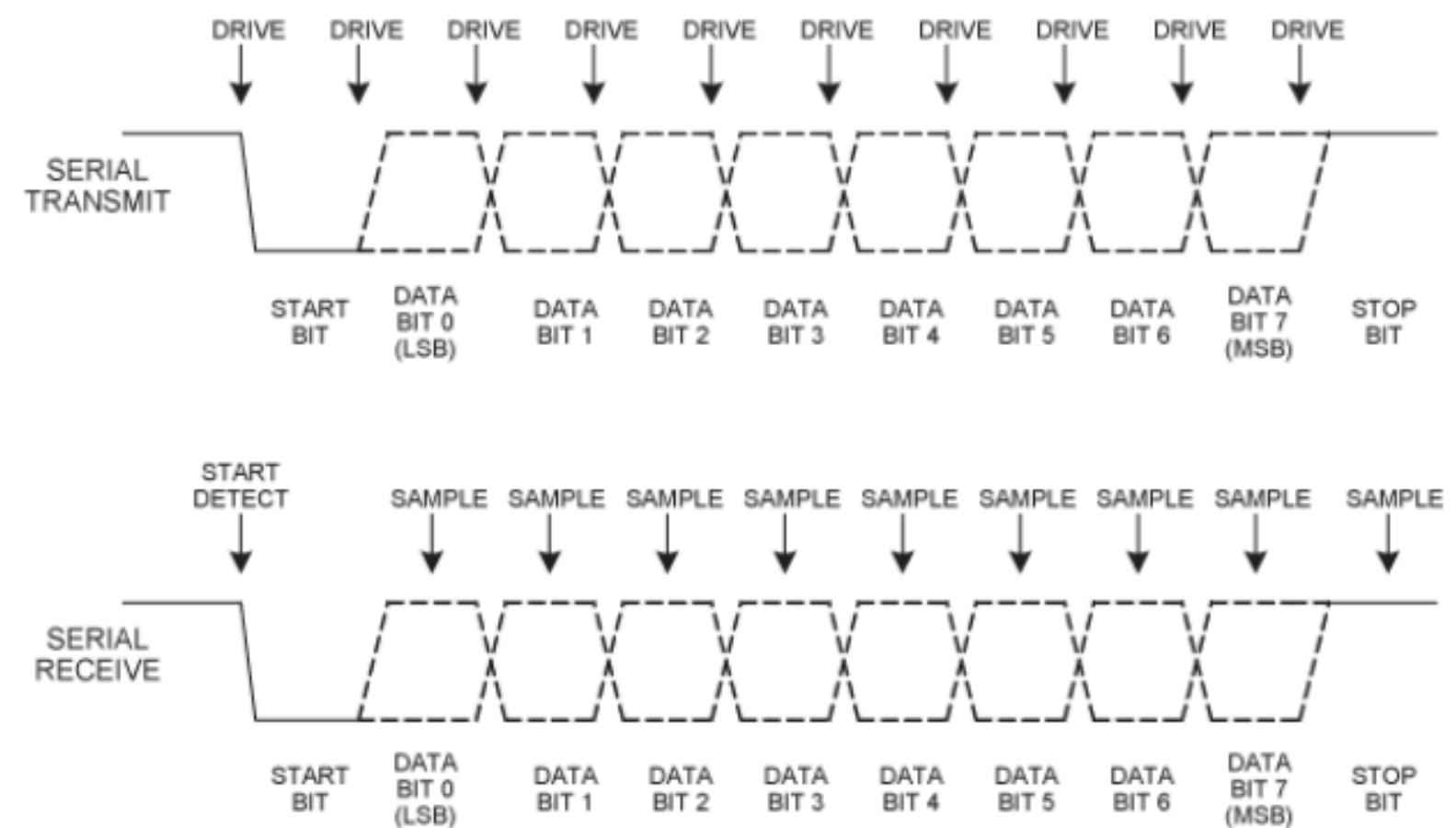
- Bluetooth 모듈 ↔ MCU 데이터 송·수신
- 명령어 및 상태 메시지 전달

(3) 동작 방식(도식 설명)

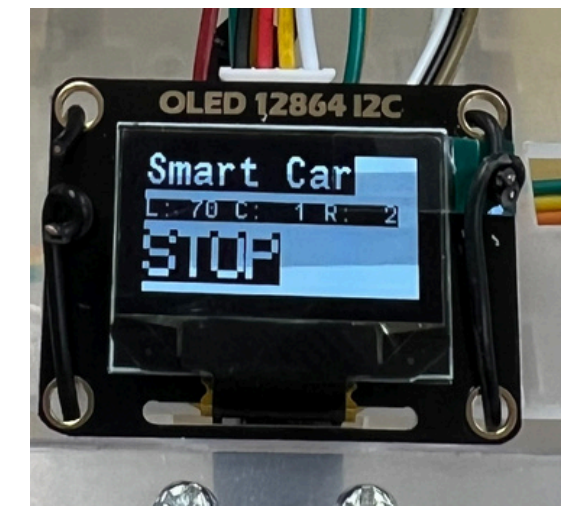
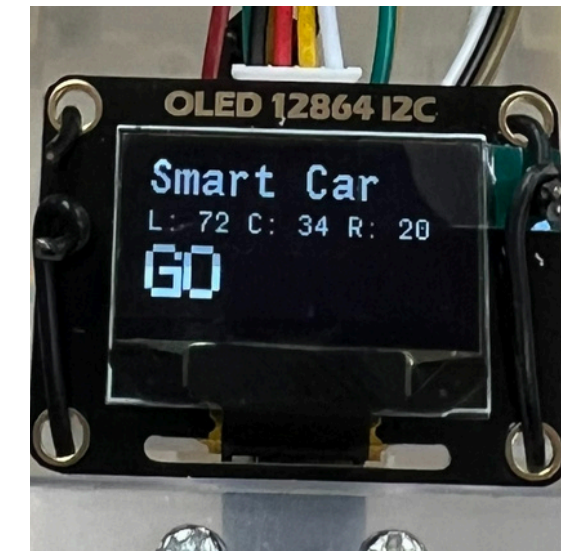
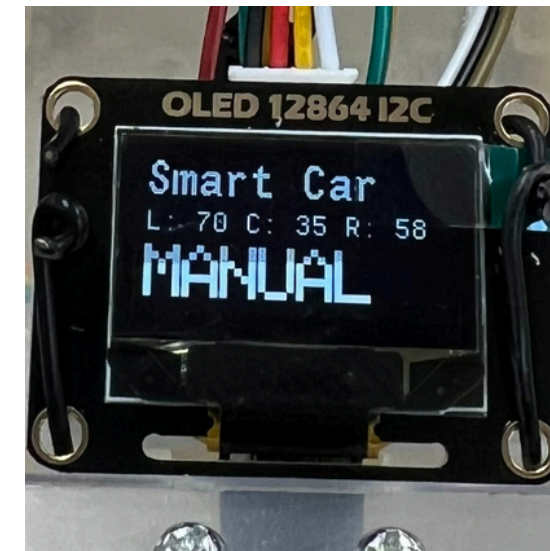
- TX/RX 2개 라인 사용
- Start Bit / Data Bit / Stop Bit 구조로 전송
- 별도 클럭 없이 Baud Rate 사전 설정

(4) 특징 및 장단점

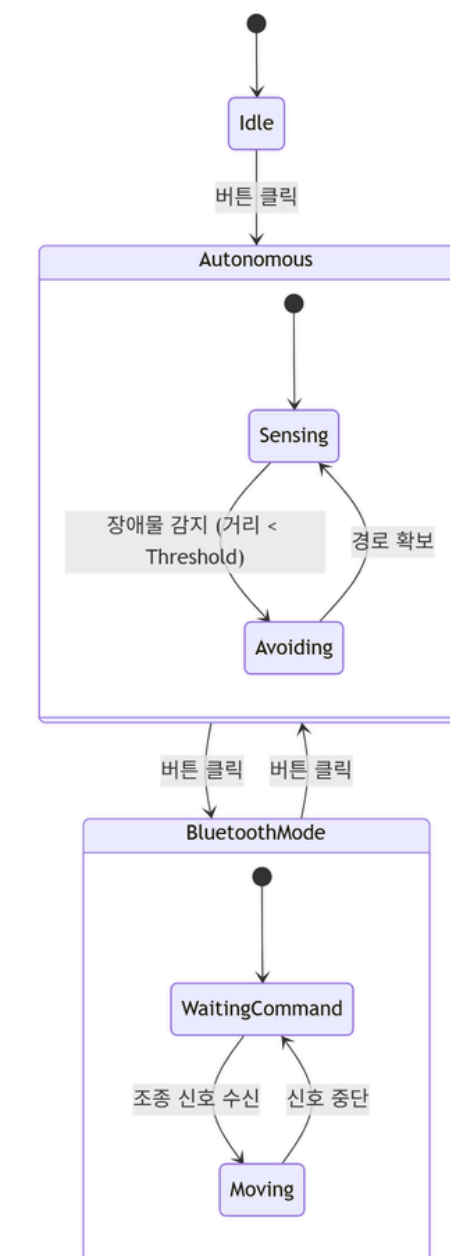
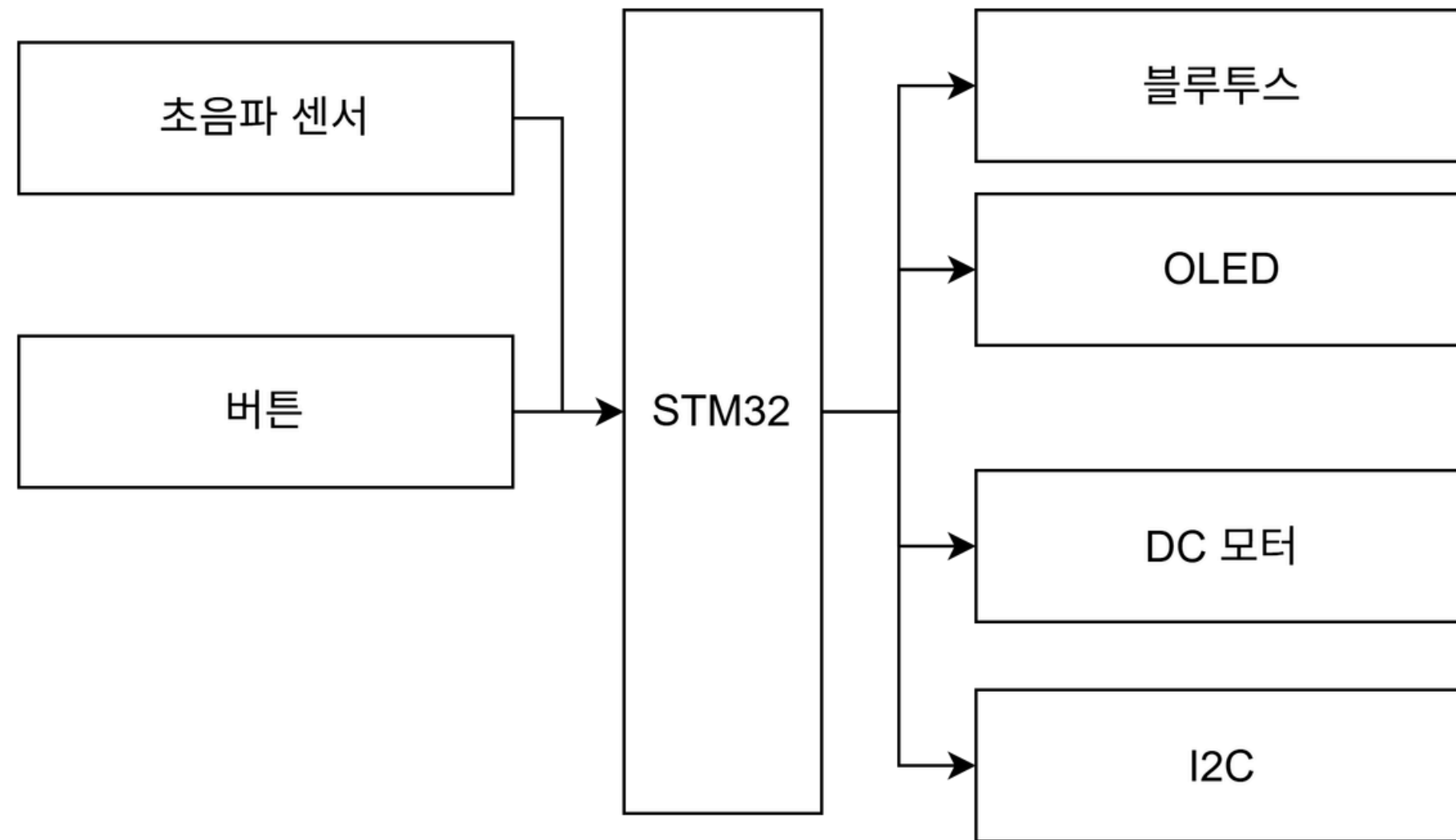
- 장점 : 구현 단순, 디버깅 용이, 소형 임베디드에 적합
- 단점 : 다수 장치 연결 한계, 통신 속도 제한



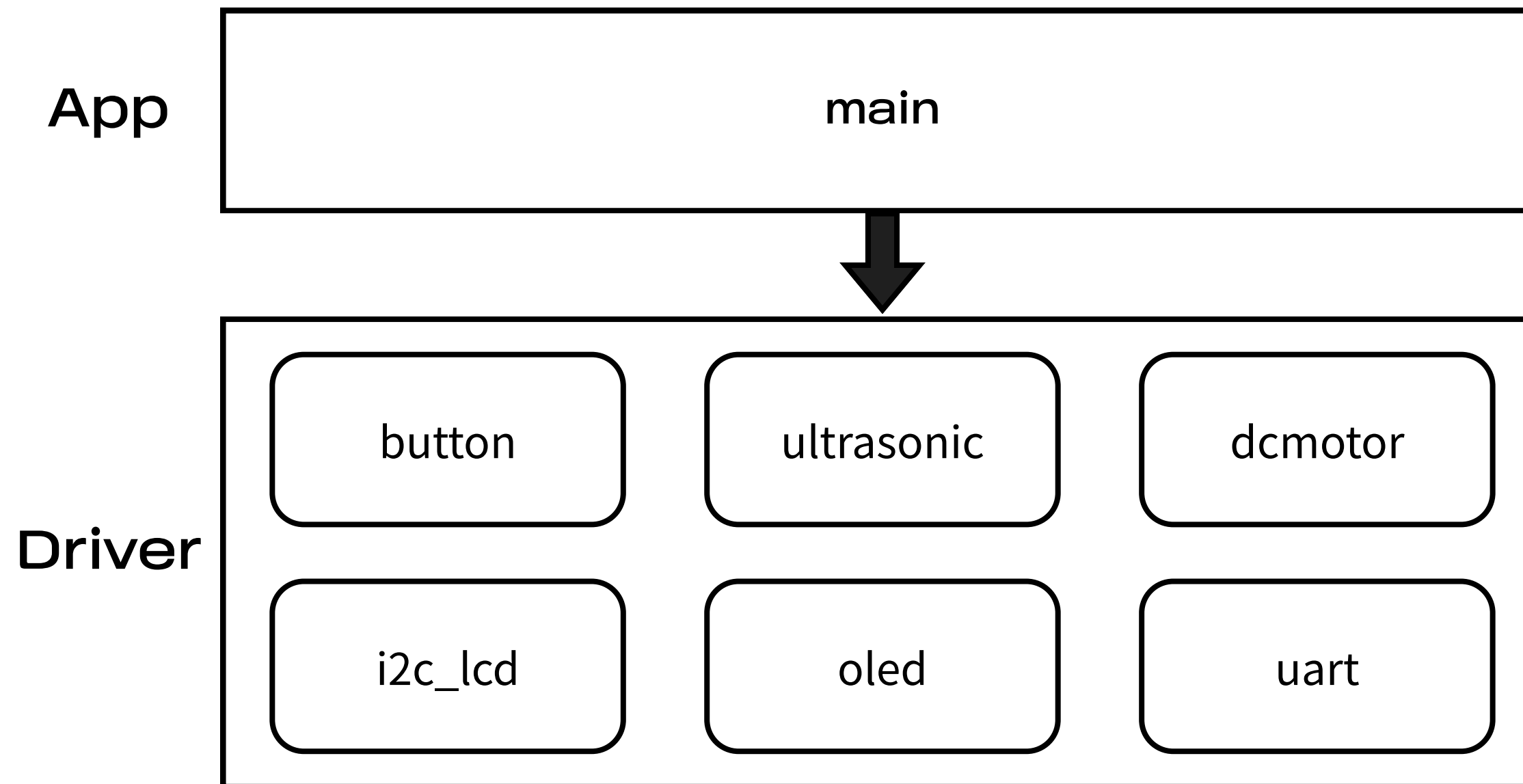
요소 기술: LCD/OLED



S/W 기본 설계



전체 펌웨어 구조



RTOS

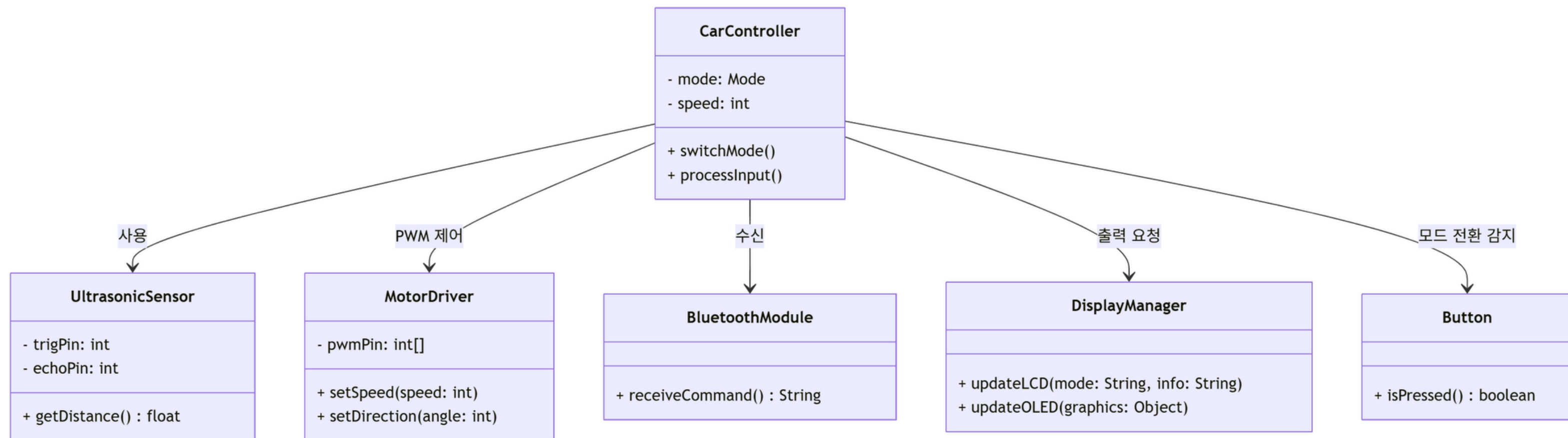
(1) 개요

- RTOS (Real-Time Operating System) : 정해진 시간 안에 여러 작업을 안정적으로 처리하기 위한 실시간 운영체제

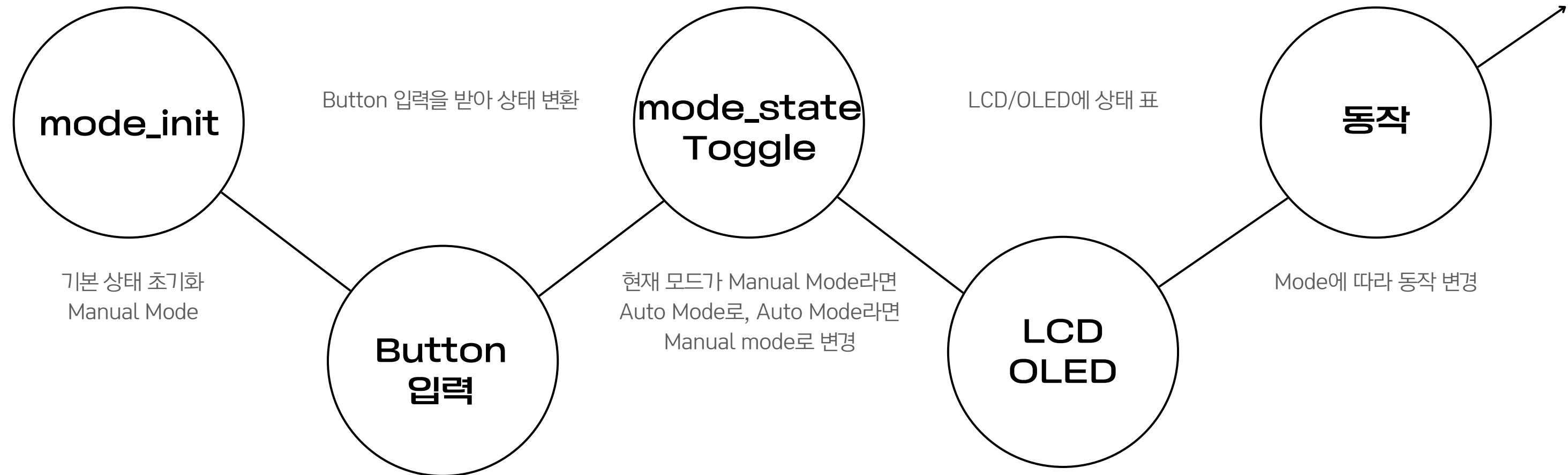
(2) 역할

- Task 관리:
 - 여러 기능을 독립적인 작업 단위로 나누어 실행
- Scheduling:
 - 우선순위에 따라 어떤 Task를 먼저 실행할지 결정
- Timing 제어:
 - 주기적인 센서 측정, 통신 처리, 출력 제어를 일정한 시간 기준으로 수행
- Resource 관리:
 - Queue, Semaphore, Mutex 등을 활용해 Task 간 데이터 공유와 충돌을 제어

RTOS Task 구조



Mode



Mode

Manual Mode <-> Auto Mode

dcmotor.c

```
18  /* ===== AUTO MODE STATE ===== */
19  uint8_t auto_mode_state = 0;

→ Manual / Auto 상태를 결정하는 전역 상태 변수

140 /* ===== MODE CHANGE BY BUTTON ===== */
141 void mode_check()
142 {
143     if (get_button(BUTTON0_GPIO_Port, BUTTON0_Pin, BUTTON0) == BUTTON_PRESS)
144     {
145         auto_mode_state = !auto_mode_state;
146
147         move_cursor(0,0);
148         if (auto_mode_state)
149         {
150             lcd_string("AUTO Mode      ");
151         }
152         else
153         {
154             lcd_string("Manual Mode   ");
155             stop();
156         }
157         move_cursor(1,0);
158         lcd_string("                ");
159     }
160 }
```

→ BUTTON0 클릭 시

→ auto_mode_state 토글

→ Manual ↔ Auto 변환 발생

```
731 void StartTask02(void *argument)
732 {
733     for(;;)
734     {
735         mode_check(); // BUTTON0 클릭 시 auto_mode_state 토글
736         osDelay(1);
737     }
738 }
```

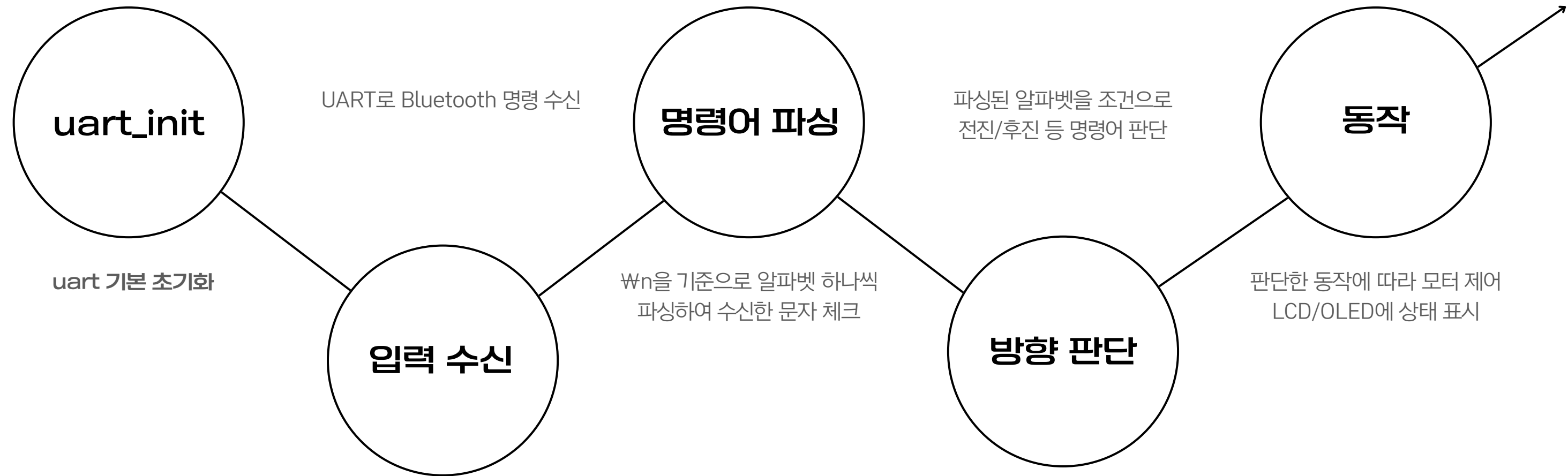
→ RTOS Task(myTask02) 에서 주기적으로 모드 변경 감시

```
695 void StartTask01(void *argument)
696 {
697     extern uint8_t auto_mode_state;
698     for(;;)
699     {
700         // 자율 주행 모드가 아닐 때만 블루투스 명령 실행 (모터 제어권 충돌 방지)
701         if (auto_mode_state)
702         {
703             auto_drive(); // 자율 주행 로직 실행
704         }
705         if (!auto_mode_state)
706         {
707             manual_mode_run();
708         }
709         osDelay(1);
710     }
711 }
```

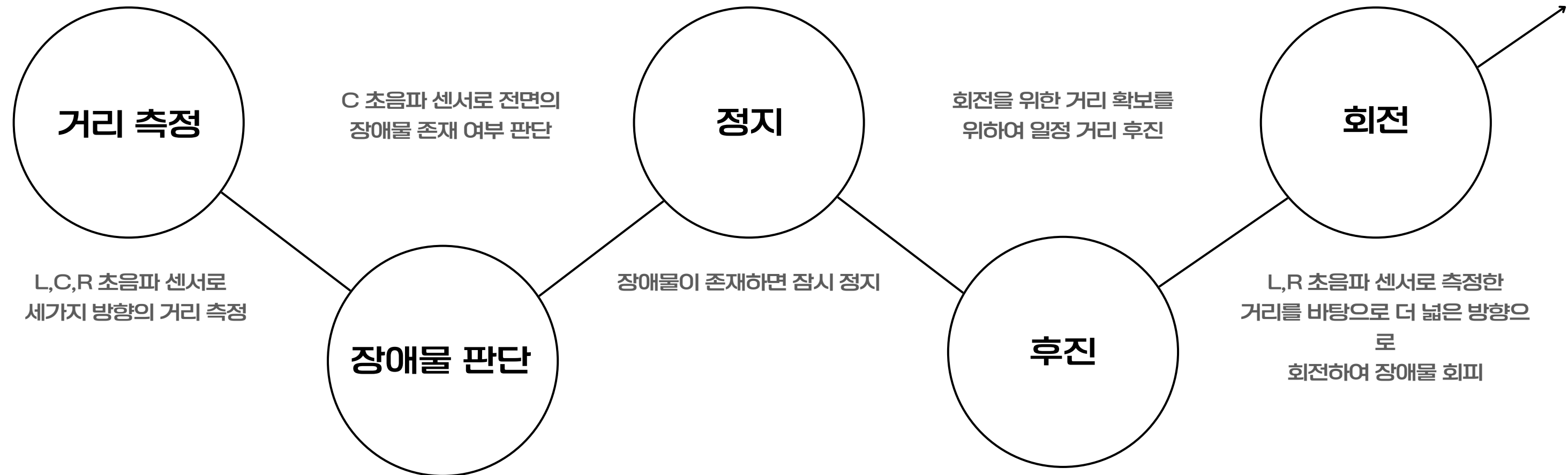
→ auto_mode_state 값에 따라

→ Manual Mode / Auto Mode 동작 분기

Manual Mode



Auto Mode



Auto Mode

main.c

```
694  /* StartTask01 (Manual Mode) 수정 */
695  void StartTask01(void *argument)
696  {
697      extern uint8_t auto_mode_state;
698      for(;;)
699      {
700          if (auto_mode_state)
701          {
702              auto_drive();
703          }
704          if (!auto_mode_state)
705          {
706              manual_mode_run();
707          }
708          osDelay(1);
709      }
710  }
```

→ Auto Mode일 때만 auto_drive() 실행
→ Manual / Auto 주행 로직 분리

dcmotor.c

```
18  /* ===== AUTO MODE STATE ===== */
19  uint8_t auto_mode_state = 0;
    → Manual / Auto 상태를 결정하는 전역 상태 변수

162 /* ===== AUTO DRIVE LOGIC ===== */
163 void auto_drive(void)
164 {
165     if (!auto_mode_state) return; → Auto Mode가 아니면 즉시 리턴
166                                     → 자율 주행 보호 조건
167     int base_speed = 60;
168     int turn_speed = 60;
169     int limit = 15; → 기본 주행 속도 / 회피 속도 / 장애물 거리 기준 설정
```

```
171  if (distance_center > 0 && distance_center <= limit)
172  {
173      stop(); osDelay(200);
174      backward(base_speed); osDelay(600);
175      stop(); osDelay(200);
176
177      int l = (distance_left <= 0) ? 999 : distance_left;
178      int r = (distance_right <= 0) ? 999 : distance_right;
179
180      if (l > r) turn_left_only_back(turn_speed);
181      else      turn_right_only_back(turn_speed);
182
183      osDelay(500);
184      stop(); osDelay(200);
185  }
    → 전방 장애물 감지
    → 후진 → 좌/우 거리 비교 → 더 넓은 방향으로 회피

186  else if (distance_right > 0 && distance_right <= limit)
187  {
188      turn_left_only_back(turn_speed);
189  }
190  else if (distance_left > 0 && distance_left <= limit)
191  {
192      turn_right_only_back(turn_speed);
193  }
194  else
195  {
196      forward(base_speed);
197  }
198  }
    → 오른쪽 장애물 → 좌회전
    → 왼쪽 장애물 → 우회전
```


Display

main.c

```
326 static void MX_I2C1_Init(void)    → 문자 LCD용 I2C 디바이스 초기화
327 {
328     hi2c1.Instance = I2C1;
329     hi2c1.Init.ClockSpeed = 100000;
330     hi2c1.Init.DutyCycle = I2C_DUTYCYCLE_2;
331     hi2c1.Init.AddressingMode = I2C_ADDRESSINGMODE_7BIT;
```

→ STM32 I2C1 설정 (100kHz, 7-bit)

i2c_lcd.c

```
72 // lcd 초기화
73 void i2c_lcd_init(void)
    → I2C 문자 LCD를 사용하기 위한 초기 설정을 수행하는 함수
...
85 // null을 만날때 까지 string을 LCD에 출력
86 void lcd_string(uint8_t *str)
    → LCD에서 문자를 출력할 행과 열 위치를 지정하는 함수
91 // 해당 줄,col으로 이동 하는 함수
92 void move_cursor(uint8_t row, uint8_t column)
    → 현재 커서 위치에 문자열을 LCD로 출력하는 함수
```

dcmotor.c

```
140 /* ===== MODE CHANGE BY BUTTON ===== */
141 void mode_check()
142 {
143     if (get_button(BUTTON0_GPIO_Port, BUTTON0_Pin, BUTTON0) == BUTTON_PRESS)
144     {
145         auto_mode_state = !auto_mode_state;
146
147         move_cursor(0,0);
148         if (auto_mode_state)
149         {
150             lcd_string("AUTO Mode    ");
151         }
152         else
153         {
154             lcd_string("Manual Mode  ");
155             stop();
156         }
157         move_cursor(1,0);
158         lcd_string("                ");
159     }
160 }
```

→ Manual ↔ Auto 전환 시 I2C LCD에 상태 표시

Display

oled_ui.c

```
15 void oled_display_main(void)
16 {
17     char buf[32];
18     static uint8_t blink = 0;    // ★ 점멸 토글 변수
19
20     blink ^= 1;    // 호출될 때마다 0 ↔ 1
21     ssd1306_Fill(Black);
22
23     /* AVOID / STOP 상태에서만 점멸 반전 */    → GO 제외한 상태에서 경고 점멸
24     if (auto_mode_state && distance_center < WARN_DIST)
25     {
26         if (blink)
27         {
28             ssd1306_InvertRectangle(
29                 0, 0,
30                 SSD1306_WIDTH - 1,
31                 SSD1306_HEIGHT - 1
32             );
33         }
34     }
35
36     ssd1306_SetCursor(0, 0);
37     ssd1306_WriteString("Smart Car", Font_11x18, White);
38
39     sprintf(buf, "L:%3d C:%3d R:%3d",    → OLED 상단에 시스템 제목 표시
40         distance_left,
41         distance_center,
42         distance_right);    → 초음파 L, C, R 출력
43     ssd1306_SetCursor(0, 22);
44     ssd1306_WriteString(buf, Font_7x10, White);
45
46     ssd1306_SetCursor(0, 36);
```

```
48     if (!auto_mode_state)
49     {
50         ssd1306_WriteString("MANUAL", Font_16x24, White);    → Manual Mode
51     }
52     else if (distance_center < STOP_DIST)
53     {
54         ssd1306_WriteString("STOP", Font_16x24, White);    → 전방 거리 < STOP_DIST
55     }    "STOP" 출력
56     else if (distance_center < WARN_DIST)
57     {
58         ssd1306_WriteString("AVOID", Font_16x24, White);    → 전방 거리 < WARN_DIST
59     }    "AVOID" 출력
60
61     ssd1306_SetCursor(0, 58);
62     if (distance_left > distance_right)
63         ssd1306_WriteString("LEFT", Font_7x10, White);
64     else
65         ssd1306_WriteString("RIGHT", Font_7x10, White);
66
67     {
68         ssd1306_WriteString("GO", Font_16x24, White);    → 장애물 없음
69     }    "GO" 출력
70
71     ssd1306_UpdateScreen();    → 내부 버퍼에 그린 내용을 OLED 패널에 출력
72 }
```

main.c

```
749 void StartTask03(void *argument)
750 {
751     ssd1306_Init();
752     for(;;)
753     {
754         oled_display_main();
755         osDelay(200);    → OLED 초기화는 RTOS Task에서 1회 수행
756     }    → 주기적으로 OLED 화면 갱신 (UI Task 분리)
757 }
```

Troubleshooting

현상

- 자율 주행 중 장애물 회피 시 충분히 회전하지 못함
- 회전 후 벽에 재충돌하는 현상 발생

원인

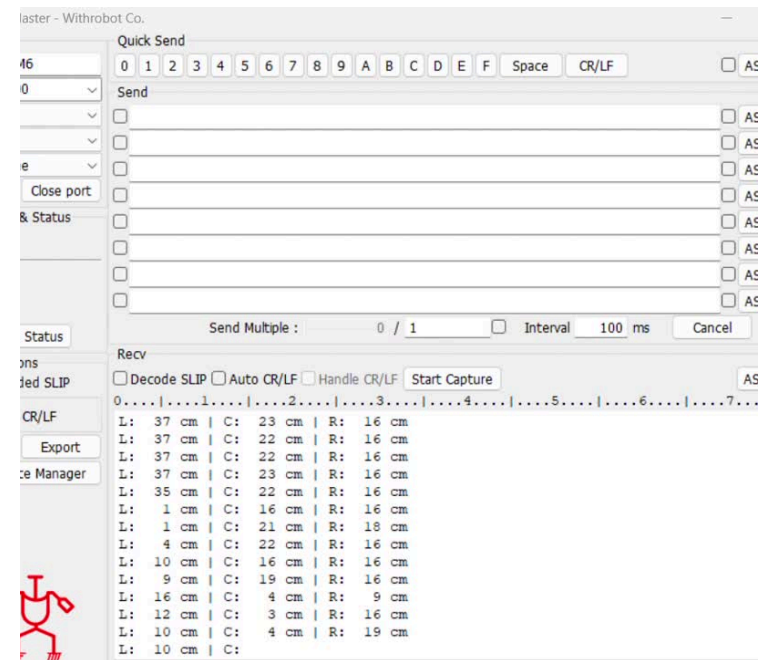
- 회전 시 모터 속도와 회전 유지 시간이 부족함
- 회전 직후 다음 동작으로 넘어가 자세 안정 시간이 없음

조치

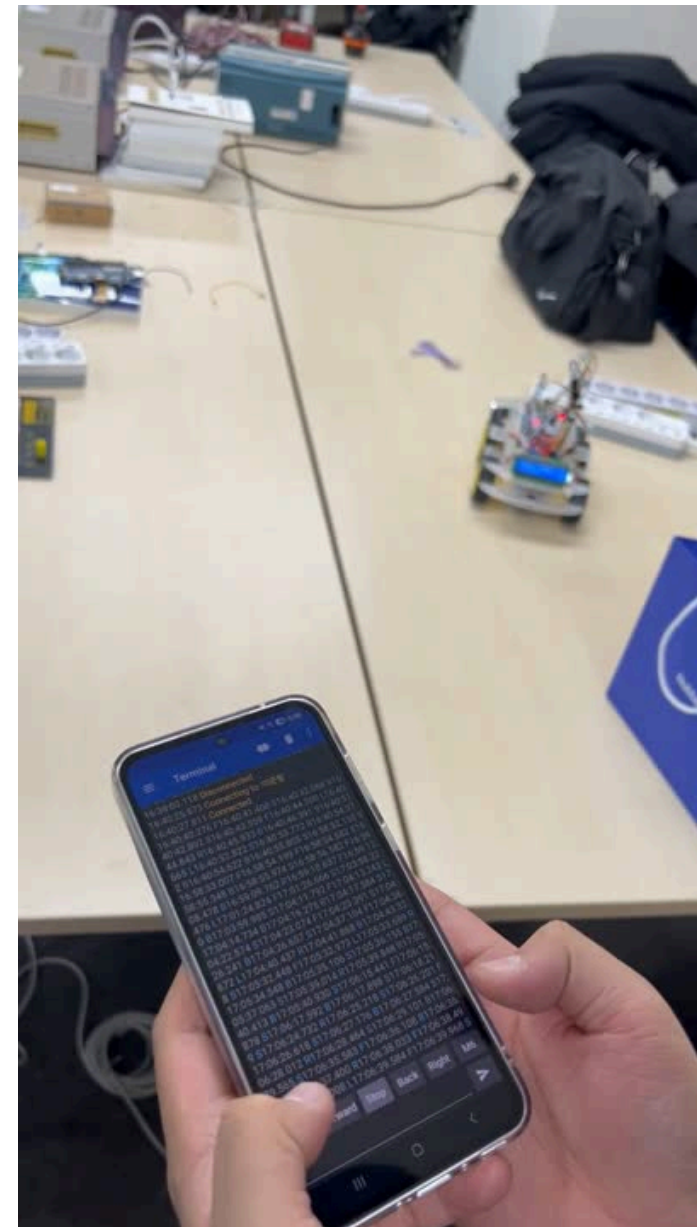
- 속도 파라미터 분리
→ fast_speed(직진), back_speed(후진), turn_speed(회전)로 분리
- 유효 거리 값 보정 추가
→ 센서 값 ≤ 0 인 경우 999로 치환
- 전방 장애물 회피 로직 강화
→ 후진 시간 증가 + 회전 유지 시간 추가
- 측면 장애물 처리 시간 증가
→ 회전 후 유지 시간(osDelay) 추가
- 정상 주행 속도 변경 → forward(fast_speed)로 변경



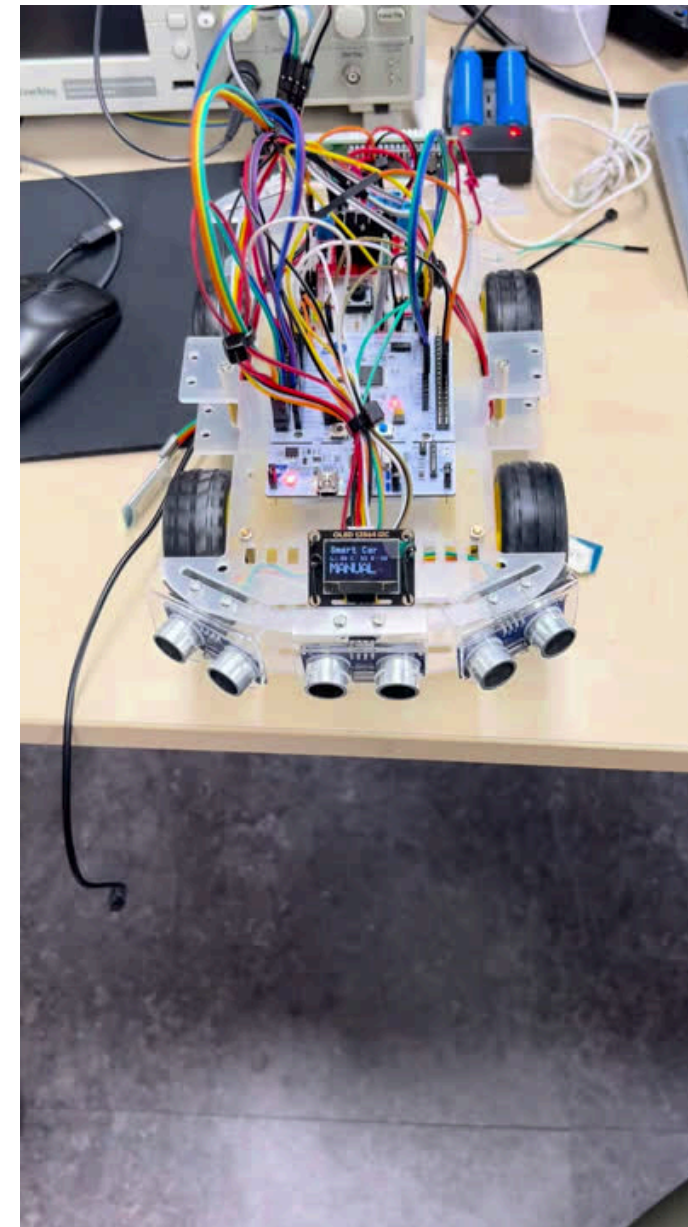
동작 영상



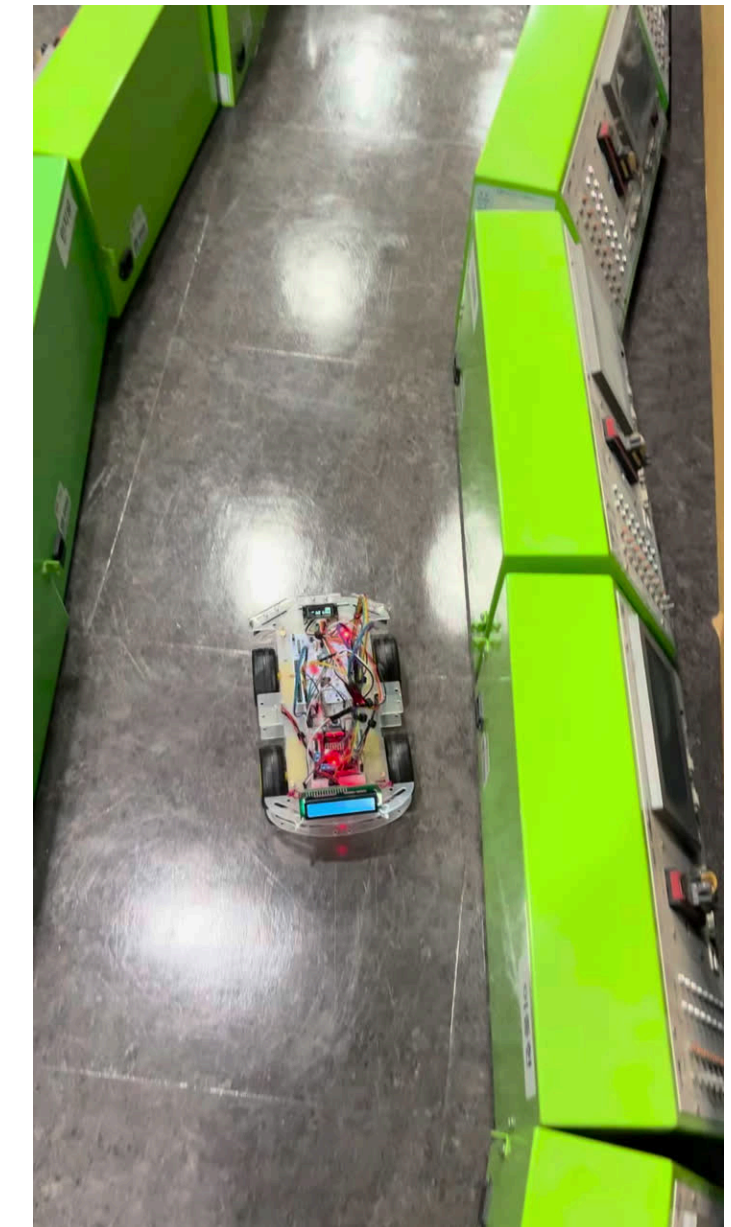
초음파 센서 동작



Manual mode 주행

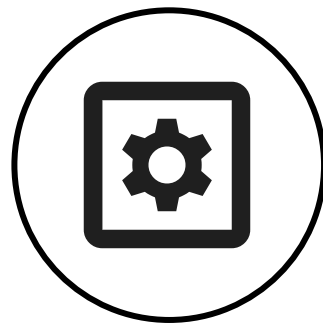


Auto mode 동작



Auto mode 주행

펌웨어 개발을 통해 얻은 역량



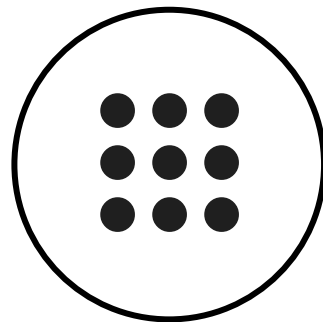
주변장치 통합 제어

STM32에서 GPIO, Timer, PWM, UART, I2C를 통합 제어



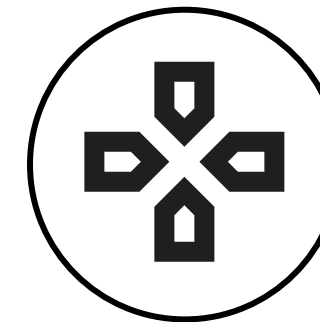
센서 기반 판단 로직

센서 입력값을 기준으로 주행 상태 판단
Manual / Auto Mode 분기 로직 구현



RTOS Task 분리

RTOS 기반 기능별 Task 분리 경험



실제 동작 기반 디버깅

실제 동작 문제를 확인하여 문제의 원인을
분석하고 파라미터를 수정